

# Linux Kernel Handling of Ethernet Devices and Network Interfaces

---

## Introduction

The Linux kernel provides a comprehensive networking subsystem that enables communication over Ethernet and other network technologies. It abstracts hardware details and offers a uniform interface for applications to send and receive data.

## 1. Network Interface Representation

In Linux, every network device is represented by a structure called `net_device`.

Each Ethernet interface (e.g., `eth0`) is registered with the kernel using `net_device`.

- The kernel maintains a list of these interfaces
- Each interface has:
  - MAC address
  - MTU
  - Flags (UP/DOWN)

## 2. Device Drivers

Ethernet communication depends on **network device drivers**:

- Drivers interact directly with hardware (NIC)
- They register themselves with the kernel networking subsystem
- Provide operations like:
  - Transmit packets
  - Receive packets
  - Handle interrupts

The driver acts as a bridge between: `H/w(NIC) ⇔ Kernel n/w stack`

### 3. Packet Handling

The Linux kernel uses a key structure:

- **sk\_buff (socket buffer)**

Represents a network packet inside the kernel

#### Packet Flow:

1. Application sends data via socket
2. Kernel creates sk\_buff
3. Passes through protocol layers (TCP/IP)
4. Sent to network device driver
5. Transmitted via Ethernet hardware

### 4. Ethernet Frame Processing

The kernel adds/removes Ethernet headers and ensures frames meet MTU requirements.

On receiving frames,

- NIC receives frame
- Driver passes it to kernel
- Kernel processes based on protocol (IP, ARP, etc.)

### 5. Networking Stack

The Linux networking stack includes Application->Socket Interface->Transport (TCP/UDP)-> IP layer-> and Network Device layers. The kernel manages encapsulation and delivery.

### 6. Configuration and Tools

User-space tools like ifconfig, ip, and ethtool interact with the kernel via Netlink to configure interfaces, assign IPs, and manage MAC addresses.

### 7. Advanced Features

The kernel supports advanced features such as traffic control, offloading (TSO/GSO), and distributed switch architecture for efficient Ethernet communication.